

Reviewer Pack — Free Cumulant Sum Gap in Read-Twice BPs for IP_2

Entry f9c38fbaabfe · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 20:25:45 UTC

Free Cumulant Sum Gap in Read-Twice BPs for IP_2

Entry ID: f9c38fbaabfe **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 20:25:45 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Read-Twice Branching Programs

Statement:

For read-twice branching programs P for IP_2 , the sum of the first $k=\lceil \log n \rceil$ free cumulants of the transition matrix entries is $\Omega(\log \text{size}(P))$, whereas for read-once BPs, this sum is $O(1)$.

Rationale (proposer's reasoning):

Free cumulants capture non-commutative structure of BP transitions; read-twice BPs exhibit richer dependencies requiring larger cumulants, while read-once BPs have simpler, bounded cumulant behavior.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ea4ad7e198827b1c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_read_twice_bp(n):
        # Generate a read-twice branching program for IP_2 with n states
        bp = [[random.choice([0, 1]) for _ in range(2)] for _ in range(n)]
        return bp

    def transition_matrix(bp):
        n = len(bp)
        T = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                if bp[i][0] == 1:
                    T[j][(i + 1) % n] += 1
                if bp[i][1] == 1:
                    T[(j + 1) % n][i] += 1
        return T

    def free_cumulants(T):
        # R-transform inversion formula to compute free cumulants (simplified)
        n = len(T)
        det_T = determinant(T)
        if det_T == 0:
            return [float('inf')] * n
        inv_T = inverse_matrix(T, det_T)
        cumulants = []
        for i in range(n):
            cumulant = 0
            for j in range(n):
                cumulant += T[i][j] * inv_T[j][i]
            cumulants.append(cumulant)
        return cumulants

    def determinant(matrix):
        n = len(matrix)
        if n == 1:
            return matrix[0][0]
        det = 0
        for j in range(n):
            minor = [row[:j] + row[j+1:] for row in matrix[1:]]
            det += ((-1) ** j) * matrix[0][j] * determinant(minor)
```

```

    return det

def inverse_matrix(matrix, det):
    n = len(matrix)
    inv = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            minor = [row[:j] + row[j+1:] for row in matrix[:i] + matrix[i+1:]]
            inv[i][j] = ((-1) ** (i+j)) * determinant(minor) / det
    return inv

def ceil_log_n(n):
    return math.ceil(math.log2(n))

n = random.randint(5, 40)
bp = generate_read_twice_bp(n)
T = transition_matrix(bp)
cumulants = free_cumulants(T)
k = ceil_log_n(n)
sum_first_k_cumulants = sum(cumulants[:k])

size_P = len(bp) ** 2
if sum_first_k_cumulants < math.log(size_P):
    conjecture_holds = False
    counterexample = "sum_first_k_cumulants < log(size(P))"
else:
    conjecture_holds = True
    counterexample = ""

return {
    "metric_name": "sum_first_k_cumulants",
    "metric_value": sum_first_k_cumulants,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"sum_first_k_cumulants < log(size(P))\" first_failing_seed={first_failing_seed}")

```

```
else:
    print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction} < 0.8")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support/falsification. |
next: Increase timeout duration and re-run test with identical parameters

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	61422
2	preregistration	ollama_remote	qwen3:8b	0	0	24097
3	novelty	ollama_remote	qwen3:8b	0	0	21274
4	novelty	ollama_remote	qwen3:8b	0	0	14397
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17966
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11377
7	judge	ollama_remote	qwen3:8b	0	0	16302

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 166836 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/f9c38fbaabfe.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f9c38fbaabfe.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/f9c38fbaabfe.tar.gz (if generated)